Spring Boot Essentials

600 × 400

3-Day Workshop: Building Scalable APIs

Training Roadmap

Day 1: Fundamentals

- Spring Boot Setup & Auto-Config
- Inversion of Control (IoC)
- Dependency Injection (@Autowired)
- Bean Scopes (Singleton vs Prototype)

Day 2: REST API

- REST Principles & Methods
- @RestController & @RequestMapping
- GET: Retrieving Data (@PathVariable)
- POST: Creating Data (@RequestBody)

Day 3: Config & Testing

- Properties & @Value
- Profiles (Dev/Prod)
- Postman Testing
- Final Project

Day 2

Building the REST API Layer

Day 2

In **Spring Boot / Spring MVC**, the `@RestController` annotation is essentially a **combination** of two other annotations:

@RestController = @Controller + @ResponseBody

Breakdown

- `@Controller`
  - Marks a class as a Spring MVC controller.
  - Used to define request-handling methods.
  - By default, it returns **views** (e.g., JSP, Thymeleaf) unless `@ResponseBody` is used.
- `@ResponseBody`
  - Indicates that the return value of a method should be **written directly to the HTTP response body** instead of rendering a view.
  - Automatically serializes Java objects to JSON or XML (depending on `Content-Type`).
- `@RestController`
  - Combines both behaviors:
    - Marks the class as a controller.
    - Ensures **all methods** return data directly (JSON/XML) without needing `@ResponseBody` on each method.

Day 2

Without @RestController

```
@Controller
public class MyController {

    @GetMapping("/hello")
    @ResponseBody
    public String hello() {
        return "Hello World";
    }
}
```

With @RestController

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class MyRestController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello World"; // Automatically JSON or plain text
    }
```

Day 2

In Spring, the `@RestController` annotation is a specialized version of the `@Controller` annotation.
It is used to create RESTful web services and simplifies the controller implementation by combining `@Controller` and `@ResponseBody` annotations.

The `@Controller` annotation is used to mark a class as a Spring MVC controller. It is a specialization of the `@Component` annotation, allowing Spring to auto-detect the class through classpath scanning. Typically, it is used in combination with `@RequestMapping` to handle HTTP requests.

Module 6: REST Controllers

@RestController

Combines @Controller and @ResponseBody. Returns domain objects directly as JSON.

@RequestMapping

Defines the base URL path for the controller.

```
@RestController
@RequestMapping("/api/books")
public class BookController { ... }
```

Module 8: GET Requests & Paths

Get All Resources

Use @GetMapping to fetch data.

```
@GetMapping
public List getAll() {
    return List.of(new Book("Java 101"));
}
```

Get Single Resource

Use @PathVariable to extract ID from URL.

```
@GetMapping("/{id}")
public Book getOne(@PathVariable String id) {
    return service.find(id);
}
```

Exercise 8.1: Implementing GET

Hands-on Activity

Objective: Create endpoints to retrieve data.

- Create BookController with @RequestMapping("/api/books").
- Implement GET /api/books that returns a static list of 3 books.
- Implement GET /api/books/{id}.
- If the ID matches one of the books, return it. Else, return null (or throw exception).
- Test in browser: http://localhost:8080/api/books.

Module 8: POST Requests & Body

Creating Resources

- Use @PostMapping for creation.
- Use @RequestBody to map incoming JSON to a Java Object.
- Ideally return HTTP 201 (Created) status.

```
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public Book create(@RequestBody Book book) {
    return service.save(book);
}
```

Exercise 8.2: Implementing POST

+Hands-on Activity 2:45 - 4:00 (75 min)

Objective: Allow clients to add new books.

- Add a POST endpoint to your controller.
- Accept a Book object using @RequestBody.
- Add this new book to your static list (simulate a database save).
- Return the saved book object.

@PathVariable

Used for:

✔ Resource IDs
✔ REST-style URLs

Example:

```java
@GetMapping("/{id}")
public User getUser(@PathVariable Long id) {
    return service.findById(id);
}
```

Call:

```bash
GET /api/users/10
```

Exercise 8.3: Implementing @PathVariable

+Hands-on Activity

Objective: use @PathVariable to get books.

@RequestParam

Used for:

- Filters
- Pagination
- Sorting

```java
@GetMapping("/search")
public List<User> search(
  @RequestParam String name,
  @RequestParam int page
)
```

Call:

```ruby
?name=Rey&page=1
```

Exercise 8.4: Implementing @RequestParam

+Hands-on Activity 2:45 - 4:00 (75 min)

Objective: get a specific book using @RequestParam.

@ResponseBody

Old style:

```java
@ResponseBody
@GetMapping("/hello")
public String hello() {
  return "Hi";
}
```

Modern:

```css
@RestController
```

Exercise 8.5: Implementing @ResponseBody

+Hands-on Activity

Objective: get all books and display using @ResponseBody.

@RequestBody

Used for JSON payload.

```java
@PostMapping
public User create(
  @RequestBody UserDTO dto
)
```

Converts:

```javascript
JSON → Java Object
```

DTO - Data Transfer Object

- **DTO is the object you use to send and receive data in your API, instead of using your database entity.**
- **DTO exists to separate your API from your database.**

Without DTO:

```arduino
Client ↔ Entity ↔ Database   ❌ (Bad Practice)
```

With DTO:

```arduino
Client ↔ DTO ↔ Service ↔ Entity ↔ Database   ✅
```

Exercise 8.6: Adding DTO

+Hands-on Activity

Objective: Know the difference of DTO and Entity

- Add a BookDTO.
- use it replacing Book to return a requested Object.